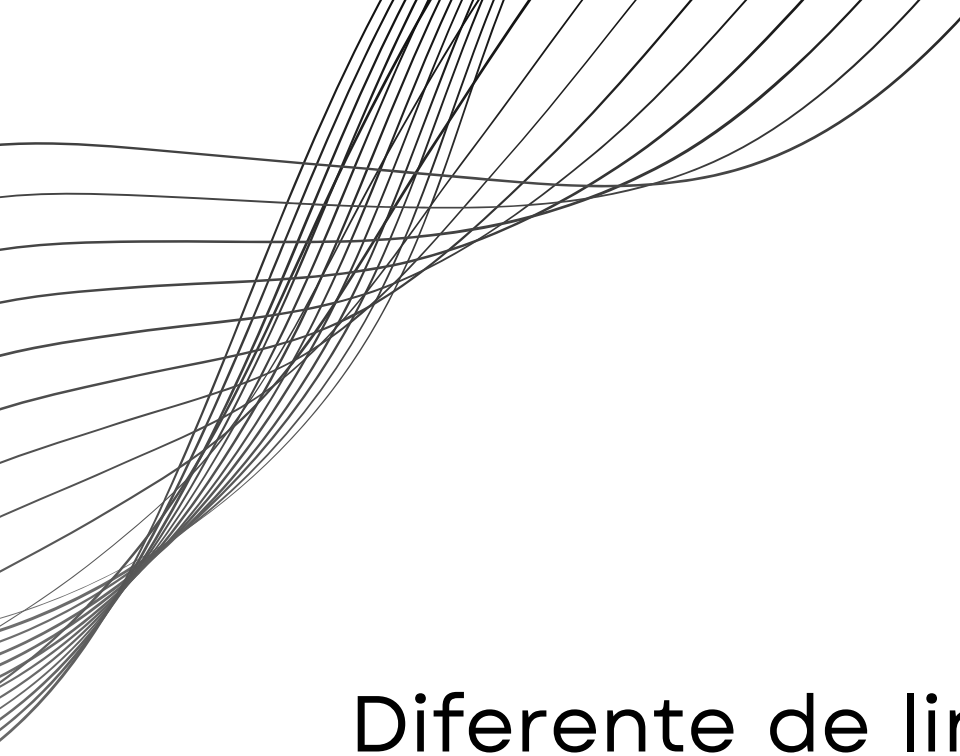


POO COM PYTHON

Pedro Henrique Ribeiro Dias

Inatel



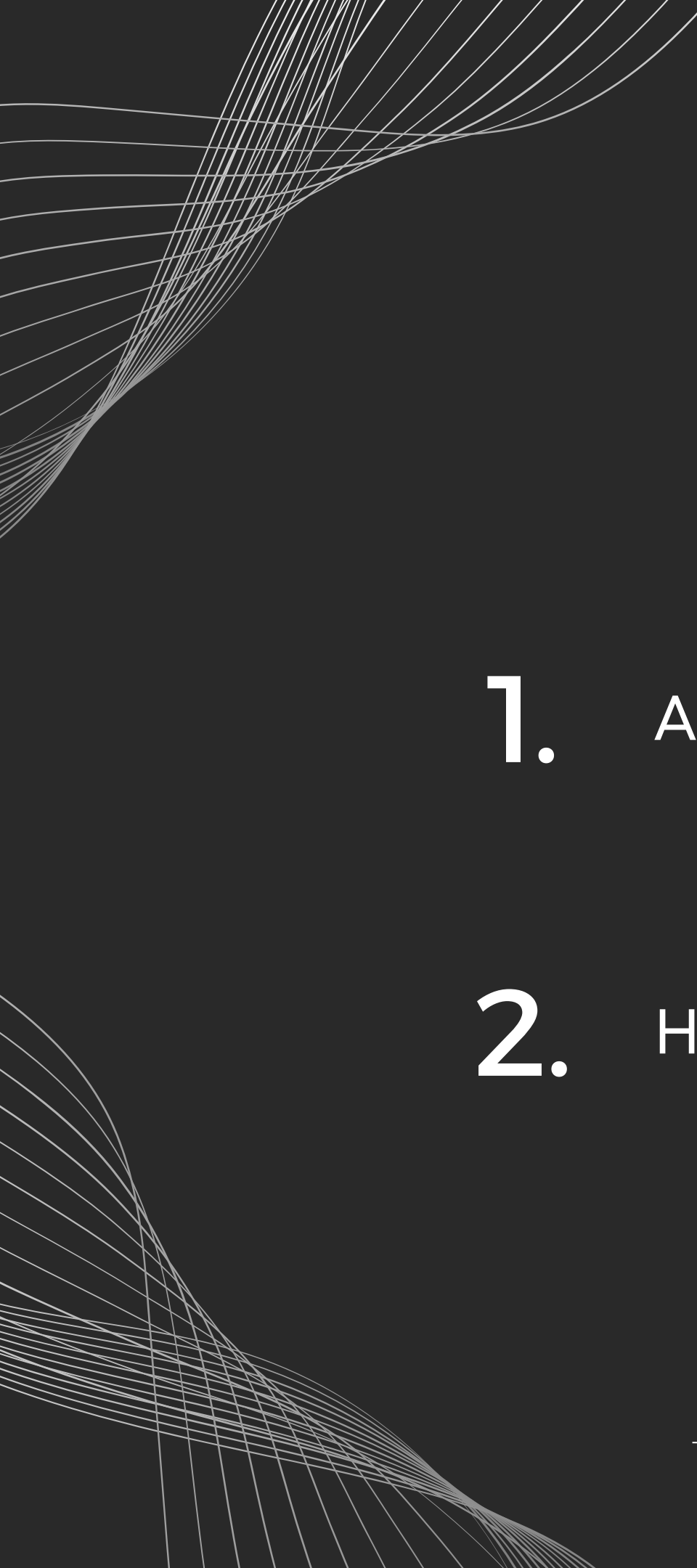
INTRODUÇÃO

Diferente de linguagens que sobrecarregam o desenvolvedor com sintaxes rígidas e gerenciamento manual complexo, o Python adota a filosofia de que "legibilidade conta". Na Programação Orientada a Objetos, isso se traduz em um código limpo, intuitivo e extremamente poderoso.

- Gerenciamento Inteligente: Assim como as linguagens modernas, o Python utiliza o Garbage Collector para cuidar da memória, permitindo que você foque na arquitetura do software.

POO COM PYTHON

1. Permite melhor organização, reutilização de código e manutenção facilitada, promovendo sistemas altamente escaláveis e flexíveis.
2. Criamos classes como moldes para objetos que possuem atributos e métodos, ou seja, características e comportamentos



PILARES DA POO

1. Abstração

2. Herança

3. Encapsulamento

4. Polimorfismo

AGREGAÇÃO

Agregação (Relacionamento "Tem"):

- É um tipo de Composição mais fraco.
- Representa uma relação "todo-parte" onde as partes podem existir independentemente do todo.
- Exemplo: Um Carro agrega um Motor. O motor pode ser removido e usado em outro carro.

COMPOSIÇÃO

Composição (Forte): A parte não pode existir sem o todo.

- Exemplo: Um Carro compõe um Chassi. Se o carro for destruído, o Chassi não sobrevive sozinho.

CONSTRUTORES E COLECTIONS

1. Construtores:
 - Método automático que inicializa o objeto.
 - Garante que o objeto nasça em um estado válido.
 - Chamado usando new.
2. Coleções (Listas e Vetores):
 - Usadas para armazenar vários objetos.
 - List<T>: Coleção dinâmica (tamanho variável).
 - Polimorfismo: Permite armazenar objetos de diferentes classes filhas em uma lista do tipo da classe mãe (List<Pai>).

INTERFACES

- Interfaces definem métodos obrigatórios que as classes concretas devem implementar. Servem para garantir consistência e flexibilidade no código.

Em Python, usamos classes abstratas para representar interfaces. E em Python são criadas utilizando o módulo abc

INTERFACE

1. Criamos uma classe que herda de ABC
2. Definimos métodos com o decorador @abstractmethod
3. As classes concretas implementam esses métodos
4. Exemplo herança de interface (interface de ação):
 - a. class Guerreiro(IAcao)
 - Obriga o guerreiro a implementar o método que a interface ação carregar.

POR QUE USÁ-LAS?

- Garantem que diferentes classes, porém que possuem comportamentos semelhantes, compartilhem o mesmo “contrato” para que sejam obrigadas a implementá-lo.
- Facilitam a substituição de implementações.
- Tornam o código mais modular e escalável.
- Permitem o uso de polimorfismo de forma limpa.

INTERFACES E RELAÇÕES

- Nosso exemplo prático implementa um mini- sistema de RPG que demonstra todos os conceitos de POO em Python:
- Interface (IAcao) → define o contrato para métodos atacar() e defender(). Composição → cada Personagem possui uma Arma. Agregação → cada Personagem tem um Inventario que gerencia uma lista de Itens.
- Herança e Polimorfismo → classes Guerreiro e
- Mago herdam de Personagem e implementam IAcao
- de formas diferentes.
- Este exemplo conecta todos os conceitos vistos, abstração, encapsulamento, herança, polimorfismo, agregação, composição e interfaces

EXERCÍCIOS

OO com Python

Inatel

EXERCÍCIO 1

Cenário: No universo de Dark Souls, todas as entidades são consideradas um SerDasCinzas, mas um Guerreiro Solar possui o atributo exclusivo de pacto_ativo, que o diferencia nas batalhas cooperativas.

Tarefa:

- Crie uma classe base chamada SerDasCinzas.
- Defina os atributos `_vitalidade` (inteiro) e `_estamina` (inteiro) usando a convenção Python de atributos privados.
- Implemente o método `__init__` para inicializar esses dois atributos.
- Utilize o decorador `@property` para criar getters e setters controlados para a `_vitalidade`. No setter, impeça que o valor seja menor que zero.
- Crie uma classe filha chamada GuerreiroSolar que herde de SerDasCinzas.
- Adicione o atributo exclusivo `pacto_ativo` (booleano) nesta classe.
- Implemente o método `__str__` em ambas as classes para retornar uma descrição personalizada (ex: "SerDasCinzas possui 150 de vitalidade atual").
- No main, instancie um GuerreiroSolar com vitalidade 200, estamina 120 e pacto como True. Imprima o objeto no console.

EXERCÍCIO 2

Cenário: Em Overwatch, todos os agentes podem ativar seu DispositivoEspecial, mas o efeito visual e prático da habilidade varia drasticamente entre as especialidades.

Tarefa:

1. Crie uma classe base chamada `AgenteOverwatch` com um método `__init__` que receba o codinome e a categoria (Ex: Tanque, Dano, Suporte).
2. Crie um método chamado `ativar_especial()` na classe base. Use `raise NotImplementedError` para forçar que as classes filhas implementem este método.
3. Crie duas classes filhas, `AgenteTanque` e `AgenteDano`, que herdem de `AgenteOverwatch`.
4. Em ambas as classes filhas, sobrescreva o método `ativar_especial()` para imprimir uma frase que represente a ação daquela categoria (ex: "Barreira protetora ativada!" para Tanque).
5. No main, crie uma lista (list) e adicione uma instância de `AgenteTanque` (ex: Reinhardt) e uma de `AgenteDano` (ex: Tracer).
6. Utilize um laço de repetição para percorrer a lista e executar o método `ativar_especial()` de cada herói.

EXERCÍCIO 3

Cenário: O líder dos Phantom Thieves, Joker, utiliza uma Ferramenta de Infiltração (um gancho) para navegar pelos Palácios. Além disso, ele recruta Aliados de Batalha para formar seu esquadrão de suporte. Enquanto o gancho é uma ferramenta essencial e integrada ao seu equipamento (composição), os aliados são indivíduos independentes que se juntam ao time conforme a necessidade (agregação).

Tarefa:

1. Crie uma classe chamada `FerramentaInfiltracao` (Representando a Composição). Ela deve ter um atributo para o nome do item.
2. Crie uma classe chamada `AliadoDeBatalha`. Esta classe será a Agregada e deve receber em seu `__init__` o nome e o codinome do membro (ex: Morgana / Mona).

EXERCÍCIO 3

3. Crie a classe LiderJoker:

- Composição: No método `__init__`, instancie um objeto `FerramentaInfiltracao` internamente. O "gancho" deve pertencer e ser gerenciado apenas pelo Joker.
- Agregação: O construtor de `LiderJoker` deve receber como parâmetro uma lista de objetos (`List[AliadoDeBatalha]`) e atribuí-la a um atributo da classe.

4. Crie um método em `LiderJoker` chamado `exibir_esquadrão()` que itera sobre a lista de aliados agregados e imprime os detalhes de cada um.

5. No main:

- Instancie uma lista de `AliadoDeBatalha` com pelo menos dois membros (ex: Ryuji e Ann).
- Instancie o `LiderJoker`, passando a lista de aliados para ele.
- Chame o método `exibir_esquadrão()` para validar a agregação.

EXERCÍCIO 4

Cenário: Nas profundezas da infraestrutura digital de Night City, diferentes unidades (como drones, terminais e especialistas) precisam seguir um protocolo padrão para interagir com a rede. Uma Célula de Hacker coordena essas unidades para executar ataques simultâneos contra as corporações.

Tarefa:

1. A Interface Base: Crie uma classe abstrata chamada `EntidadeDeRede`.
 - Defina um método abstrato chamado `conectar_e_executar()`. Este método não deve ter implementação na classe base.
2. O Hardware (Composição): Crie uma classe chamada `ModuloNeural`.
 - No método `__init__`, ela deve ter os atributos `processamento` (ex: "Octa-Core") e `consumo_ram` (inteiro).
 - Dica: Este objeto será "parte" do hacker, ele não existe sem o seu portador.

EXERCÍCIO 4

3. Especialista (Herança e Composição): Crie a classe OperadorNetrunner que herda de EntidadeDeRede.
 - No `__init__`, o Netrunner deve compor (instanciar internamente) um objeto da classe ModuloNeural.
 - Implemente o método `conectar_e_executar()` fazendo com que o Netrunner imprima uma mensagem indicando que está quebrando o gel (ICE) de um servidor.
4. Organização (Agregação): Crie uma classe chamada ColetivoHacker.
 - Esta classe deve possuir uma lista de membros.
 - O construtor deve permitir a agregação de uma lista de objetos que sejam do tipo EntidadeDeRede.]
5. Na main:
 - Instancie um ColetivoHacker.
 - Crie e adicione múltiplos OperadorNetrunner (ex: "Spider Murphy", "Alt Cunningham") a este coletivo.
 - Demonstre o poder do grupo: percorra a lista de membros do coletivo e faça com que todos executem o método `conectar_e_executar()` ao mesmo tempo.

FIM DA AULA!



zSh3rl0cK/S01-monitoria

Contribute to zSh3rl0cK/S01-monitoria development by creating an account on GitHub.

